RolaSoft Technologies. Limited

python™



Web Development on DJANGO

Certificate in Python Django Web Application Developer

COMPUTER & WEB FUNDAMENTALS

COMPUTER & WEB FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

COMPUTER & WEB FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

PYTHON PROGRAMMING FUNDAMENTALS

PYTHON PROGRAMMING FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

PYTHON PROGRAMMING FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

LAB 1 ACTIVITY -Django Activity

🏆 Django Activity

1. Create project called rolasoft_school
2. Create app called students
3. Add a simple view returning "Welcome to Rolasoft Students Portal"
4. Connect URLs properly
5. Run server and test

🛠️ STEP 1: Create Project

Open terminal inside your **working directory**.

pip install Django

Check version:

django-admin --version

django-admin startproject rolasoft_school

Go inside the project:

cd rolasoft_school

LAB 1 ACTIVITY -Django Activity

🛠 STEP 2: Create App

Inside the project folder, run:

python manage.py startapp students

Now your structure becomes:

```
rolasoft_school/
│
├── manage.py
├── rolasoft_school/
│
└── students/
```

LAB 1 ACTIVITY -Django Activity

🛠 STEP 3: Register App in settings.py (VERY IMPORTANT)

Open:

rolasoft_school/settings.py

Find:

INSTALLED_APPS = [

Add 'students',

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'students',  # ← Add this
]
```

Save file.

LAB 1 ACTIVITY -Django Activity

🛠 STEP 4: Create the View

Open:
students/views.py

Replace content with:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Rolasoft Students Portal")
```

Save file.

LAB 1 ACTIVITY -Django Activity

🛠 STEP 5: Create App URLs File

Inside students folder, create a new file:

students/urls.py

Add this:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Save file.

LAB 1 ACTIVITY -Django Activity

🛠 STEP 6: Connect App URLs to Project URLs

Open:
rolasoft_school/urls.py

Modify it like this:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),  # Connect app
]
```

Now when **someone** visits the **homepage /,** it loads **students app**.

LAB 1 ACTIVITY -Django Activity

🛠 STEP 7: Run Migrations (Good Practice)

python manage.py migrate

🛠 STEP 8: Run Server

python manage.py runserver

You should see:

Starting development server at http://127.0.0.1:8000/

LAB 1 ACTIVITY -Django Activity

🧪 STEP 9: Test in Browser

Open:

http://127.0.0.1:8000/

You should see:

🎉 Welcome to Rolasoft Students Portal

Congratulations! Your Django project is working.

LAB 2 ACTIVITY - Login System (Authentication)

✓ **Login System (Authentication)**

Students App will support:

- ➕ Create Student
- 📄 View Students
- ✏️ Edit Student
- ❌ Delete Student
- 🔐 Login / Logout
- 🛑 Only logged-in users can access CRUD

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 1: Create Student Model

Open:

students/models.py

Replace with:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    course = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 2: Make Migrations

python manage.py makemigrations

python manage.py migrate

🛠 STEP 3: Register Model in Admin

Open:

students/admin.py

Add:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

LAB 2 ACTIVITY - Login System (Authentication)

Create superuser:

python manage.py createsuperuser

details:

Username (leave blank to use 'rolasofttechnologie'): ola

Email address: akinboder@yahoo.com

Password:

Password (again):

This password is too short. It must contain at least 8 characters.

Password:1234rola

Password (again):1234rola

Superuser created successfully.

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 4: Create Forms

Create file:
students/forms.py

Add:

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'email', 'course']
```

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 5: Create Views (CRUD + Login Protection)

Open:

students/views.py

Replace everything:

Listing: rolasoft_school/students/views.py

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 6: Configure URLs

Open:

students/urls.py

Replace with:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name='student_list'),
    path('create/', views.student_create, name='student_create'),
    path('update/<int:pk>/', views.student_update, name='student_update'),
    path('delete/<int:pk>/', views.student_delete, name='student_delete'),

    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
]
```

LAB 2 ACTIVITY - Login System (Authentication)

Open project:

rolasoft_school/urls.py

Ensure:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]
```

✅ **Configure LOGIN_URL Properly**

Open:

rolasoft_school/settings.py

Add this:

```
LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'student_list'   # or wherever you want to redirect after login
LOGOUT_REDIRECT_URL = 'login'
```

LAB 2 ACTIVITY - Login System (Authentication)

🛠 STEP 7: Create Templates

Create folder:

rolasoft_school/students/templates/students/

□ login.html

rolasoft_school/students/templates/students/login.html

```
<h2>Login</h2>
<form method="POST">
    {% csrf_token %}
    <input type="text" name="username" placeholder="Username"><br><br>
    <input type="password" name="password" placeholder="Password"><br><br>
    <button type="submit">Login</button>
</form>

{% for message in messages %}
<p style="color:red;">{{ message }}</p>
{% endfor %}
```

LAB 2 ACTIVITY - Login System (Authentication)

student_list.html

rolasoft_school/students/templates/students/student_list.html

```
<h2>Students List</h2>

<a href="{% url 'student_create' %}">Add Student</a> |
<a href="{% url 'logout' %}">Logout</a>

<hr>

{% for student in students %}
<p>
   {{ student.name }} - {{ student.course }}
   <a href="{% url 'student_update' student.pk %}">Edit</a>
   <a href="{% url 'student_delete' student.pk %}">Delete</a>
</p>
{% endfor %}
```

LAB 2 ACTIVITY - Login System (Authentication)

student_form.html

rolasoft_school/students/templates/students/student_form.html

```
<h2>Student Form</h2>

<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>

<a href="{% url 'student_list' %}">Back</a>
```

LAB 2 ACTIVITY - Login System (Authentication)

❌ **student_confirm_delete.html**

rolasoft_school/students/templates/students/student_confirm_delete.html

```
<h2>Confirm Delete</h2>

<p>Are you sure you want to delete {{ student.name }}?</p>

<form method="POST">
    {% csrf_token %}
    <button type="submit">Yes, Delete</button>
</form>

<a href="{% url 'student_list' %}">Cancel</a>
```

🛠 **STEP 8: Run Server**

python manage.py runserver

LAB 3 ACTIVITY - Modify your project

🏆 **Lab 3 – Modify your project**

Modify your project to:

1. Change TIME_ZONE to Africa/Lagos
2. Add global templates folder
3. Configure media upload
4. Configure login redirect
5. Split settings into development & production

LAB 3 ACTIVITY - SOLUTION

□ STEP 1: Change TIME_ZONE to Africa/Lagos

Open:

LAB-3/rolasoft_school/rolasoft_school/settings.py

Find:

TIME_ZONE = 'UTC'

Replace with:

TIME_ZONE = 'Africa/Lagos'

Keep:

USE_TZ = True

✓ **Done.**

LAB 3 ACTIVITY - SOLUTION

□ STEP 2: Add Global Templates Folder

1) Create Folder

Inside your project root (same level as manage.py):

rolasoft_school/

```
│
├── manage.py
├── templates/
```

2) Modify settings.py

Find the TEMPLATES section:

Change this:

'DIRS': [],

To:

'DIRS': [BASE_DIR / 'templates'],

LAB 3 ACTIVITY - SOLUTION

Full section should look like:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Now Django will search:

- App templates
- Global templates folder

✅ **Done.**

LAB 3 ACTIVITY - SOLUTION

□ STEP 3: Configure Media Upload

This allows users to upload files/images.

1) Update settings.py

Add at bottom:

MEDIA_URL = '/media/'

MEDIA_ROOT = BASE_DIR / 'media'

2) Update Project urls.py

Open:

rolasoft_school/urls.py

Add:

from django.conf import settings

from django.conf.urls.static import static

LAB 3 ACTIVITY - SOLUTION

At bottom add:

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Full example:

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

LAB 3 ACTIVITY - SOLUTION

3) Update Model Example (Optional)

If you want image upload:

image = models.ImageField(upload_to='students/')

Install Pillow:

pip install pillow

Run migrations again.

python manage.py migrate

✓ **Media configuration complete.**

LAB 3 ACTIVITY - SOLUTION

□ STEP 4: Configure Login Redirect

Open settings.py and add:

LOGIN_URL = 'login'

LOGIN_REDIRECT_URL = 'student_list'

LOGOUT_REDIRECT_URL = 'login'

Now:

- If user not logged in → redirect to login
- After login → redirect to student_list
- After logout → redirect to login

✅ **Authentication redirect configured.**

LAB 3 ACTIVITY - SOLUTION

□ STEP 5: Split Settings into Development & Production (Professional Setup)

This is how real companies structure Django.

1) Create New Folder

Inside project folder (where settings.py exists):

Create a python package name: settings :

settings/

Structure becomes:

```
rolasoft_school/
|
├── settings/
|   ├── __init__.py
|   ├── base.py
|   ├── development.py
|   └── production.py
```

LAB 3 ACTIVITY - SOLUTION

2) Move All Existing Settings into base.py

Cut everything from old settings.py

Paste into:

settings/base.py

3) Create development.py

```
from .base import *

DEBUG = True

ALLOWED_HOSTS = []
```

LAB 3 ACTIVITY - SOLUTION

4) Create production.py

```
from .base import *

DEBUG = False

ALLOWED_HOSTS = ['yourdomain.com']

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

LAB 3 ACTIVITY - SOLUTION

5) Modify manage.py

Open:

manage.py

Find:

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'rolasoft_school.settings')

Change to:

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'rolasoft_school.settings.development')

6) Modify wsgi.py & asgi.py

Change the same line to:

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'rolasoft_school.settings.production')

(Or development depending on environment)

LAB 3 ACTIVITY - SOLUTION

In the Terminal Create superuser:

python manage.py createsuperuser

details:

Username (leave blank to use 'rolasofttechnologie'): ola

Email address: akinboder@yahoo.com

Password:

Password (again):

This password is too short. It must contain at least 8 characters.

Password:1234rola

Password (again):1234rola

Superuser created successfully.

🛠 STEP 8: Run Server

python manage.py runserver

LAB 3 ACTIVITY - SOLUTION

🏗 Final Professional Structure

```
rolasoft_school/
│
├── manage.py
├── templates/
├── media/
│
├── rolasoft_school/
│   ├── urls.py
│   ├── wsgi.py
│   ├── asgi.py
│   └── settings/
│       ├── __init__.py
│       ├── base.py
│       ├── development.py
│       └── production.py
```

LAB 3 ACTIVITY - SOLUTION

🎉 What You Just Achieved

✓ Configured correct timezone for Nigeria
✓ Added global templates folder
✓ Enabled media uploads
✓ Configured login redirects
✓ Implemented professional settings separation

You now have a production-ready Django configuration structure.

LAB 3 ACTIVITY - SOLUTION

🏆 **BONUS (Industry Best Practice)**

Install:

pip install python-decouple

Use environment variables for:

- SECRET_KEY
- DEBUG
- Database passwords

Never hardcode secrets.

LAB 4 ACTIVITY

🏆 LAB 4: ACTIVITY

1. Create a page /courses/
2. Create dynamic URL /courses/<int:id>/
3. Render template instead of HttpResponse
4. Add navigation links using {% url %}

We assume your project already has an **app** called **students** from **LAB-3**.

LAB 4 ACTIVITY - SOLUTION

✅ STEP 1: Update App URLs

Open:

students/urls.py

Update it like this:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('courses/', views.course_list, name='course_list'),
    path('courses/<int:id>/', views.course_detail, name='course_detail'),
]
```

Now we have:

- /courses/
- /courses/1/
- /courses/2/

LAB 4 ACTIVITY - SOLUTION

□ STEP 2: Create Views

Open:

students/views.py

Add:

```
from django.shortcuts import render

def home(request):
    return render(request, 'students/home.html')

def course_list(request):
    # Dummy course data (later this will come from database)
    courses = [
        {"id": 1, "title": "Python Programming"},
        {"id": 2, "title": "Django Web Development"},
        {"id": 3, "title": "React Development"},
    ]
    return render(request, 'students/course_list.html', {"courses": courses})
```

LAB 4 ACTIVITY - SOLUTION

```
def course_detail(request, id):
    course = {
        "id": id,
        "title": f"Course {id}",
        "description": "This is a detailed description of the course."
    }
    return render(request, 'students/course_detail.html', {"course": course})
```

Notice:

We are using render() instead of HttpResponse.

LAB 4 ACTIVITY - SOLUTION

□ STEP 3: Create Templates

Inside:

students/templates/students/

Create the following files:

🏠 home.html

```
<h1>Welcome to Rolasoft School</h1>

<nav>
    <a href="{% url 'course_list' %}">View Courses</a>
</nav>
```

LAB 4 ACTIVITY - SOLUTION

📚 **course_list.html**

```
<h2>Available Courses</h2>

<nav>
   <a href="{% url 'home' %}">Home</a>
</nav>

<hr>

<ul>
   {% for course in courses %}
      <li>
         <a href="{% url 'course_detail' course.id %}">
            {{ course.title }}
         </a>
      </li>
   {% endfor %}
</ul>
```

LAB 4 ACTIVITY - SOLUTION

course_detail.html

```
<h2>{{ course.title }}</h2>
<p>{{ course.description }}</p>
<nav>
    <a href="{% url 'course_list' %}">Back to Courses</a> |
    <a href="{% url 'home' %}">Home</a>
</nav>
```

STEP 4: Ensure Project URLs Are Connected

Open:

rolasoft_school/urls.py

Make sure it includes:

```
from django.urls import path, include
urlpatterns = [
    path('', include('students.urls')),
]
```

LAB 4 ACTIVITY - SOLUTION

▯ STEP 5: Run Server

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-4\rolasoft_school>

python manage.py runserver

▯ TEST THESE URLs

Homepage:

http://127.0.0.1:8000/

Course List:

http://127.0.0.1:8000/courses/

Course Detail:

http://127.0.0.1:8000/courses/1/

LAB 5 ACTIVITY

🏆 **LAB-5**

Upgrade your project:

1. Add Bootstrap via CDN in base.html
2. Create reusable navbar.html
3. Add page title block
4. Add dynamic footer year using context
5. Style course list with Bootstrap cards

LAB 5 ACTIVITY - SOLUTION

□ STEP 1: Add Bootstrap via CDN in base.html

Create:

templates/base.html

Replace with this professional version:

Listing: LAB-5/rolasoft_school/templates/base.html

What we added:

- Bootstrap styling
- Page title block
- Reusable navbar
- Dynamic footer year
- Clean container layout

LAB 5 ACTIVITY - SOLUTION

□ STEP 2: Create Reusable navbar.html

Inside:
templates/navbar.html

Add:

Listing: LAB-5/rolasoft_school/templates/navbar.html

Now navbar is reusable across all pages.

LAB 5 ACTIVITY - SOLUTION

? STEP 3: Add Dynamic Footer Year Using Context

Instead of manually passing year in every view, we use a context processor (professional method).

Create **context_processors.py** inside **students app**

students/context_processors.py

Add:

Listing: LAB-5/rolasoft_school/students/context_processors.py

Register Context Processor in settings.py

Open settings.py (NOTE:base.py from lab-4) → find TEMPLATES section → inside context_processors add:

'students.context_processors.global_year',

Final part should look like:

```
'context_processors': [
    'django.template.context_processors.debug',
    'django.template.context_processors.request',
    'django.contrib.auth.context_processors.auth',
    'django.contrib.messages.context_processors.messages',
    'students.context_processors.global_year',
],
```

Now {{ year }} works in ALL templates automatically.

LAB 5 ACTIVITY - SOLUTION

✅ STEP 4: Update Home Template

Open:

students/templates/students/home.html

Update:

Listing: LAB-5/rolasoft_school/students/templates/students/home.html

Now we are using Bootstrap styling.

✅ STEP 5: Style Course List with Bootstrap Cards

Open:

students/templates/students/course_list.html

Replace with:

Listing: LAB-5/rolasoft_school/students/templates/students/course_list.html

Now your courses look professional.

LAB 5 ACTIVITY - SOLUTION

□ STEP 6: (Optional) Improve Course Detail Page

students/templates/students/course_detail.html

Update:

Listing: LAB-5/rolasoft_school/students/templates/students/course_detail.html

□ STEP 7: Run Server

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-5\rolasoft_school>

python manage.py runserver

LAB 6 ACTIVITY

🏆 LAB 6

1. Add a CSS file to style homepage
2. Add a JS file for interactivity (e.g., alert welcome message)
3. Upload profile image for student
4. Display student images in template
5. Test both static & media file rendering

LAB 6 ACTIVITY - SOLUTION

□ STEP 1: Configure Static & Media in settings.py (now: ../settings/base.py)

Make sure you have these lines:

```
# Static files
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "static files"

# Media files
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / "media"
```

LAB 6 ACTIVITY - SOLUTION

Update urls.py (../rolasoft_school/rolasoft_school/urls.py) to serve media in development:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

LAB 6 ACTIVITY - SOLUTION

□ STEP 2: Add CSS File for Homepage

1. Create folder:

static/css/

2. Create file static/css/style.css:

: lab-6/rolasoft_school/static/css/style.css

```
body {
    background-color: #f0f8ff;
    font-family: Arial, sans-serif;
}
h1 {
    color: #007bff;
    text-align: center;
    margin-top: 50px;
}
p.lead {
    text-align: center;
    font-size: 1.2rem;  }
```

LAB 6 ACTIVITY - SOLUTION

3. Include CSS in home.html:

: lab-6/rolasoft_solution/students/templates/students/home.html

```
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

LAB 6 ACTIVITY - SOLUTION

□ STEP 3: Add JS File for Interactivity

1. Create folder:

static/js/

: lab-6/rolasoft_school/static/js/

2. Create file static/js/script.js:

: lab-6/rolasoft_school/static/js/script.js:

```
window.onload = function() {
    alert("Welcome to Rolasoft School!");
}
```

3. Include JS in home.html (below <body> preferably):

```
<script src="{% static 'js/script.js' %}"></script>
```

✓ Now when you open homepage, a welcome alert should appear.

LAB 6 ACTIVITY - SOLUTION

□ STEP 4: Add Profile Image Field to Student Model

In students/models.py:

: lab-6/rolasoft_school/students/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    profile_image = models.ImageField(upload_to='students/', null=True, blank=True)

    def __str__(self):
        return self.name
```

- upload_to='students/' → images will be stored in media/students/

LAB 6 ACTIVITY - SOLUTION

Install Pillow (required for ImageField):

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-6\rolasoft_school>

pip install pillow

Run migrations:

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-6\rolasoft_school>

python manage.py makemigrations

python manage.py migrate

LAB 6 ACTIVITY - SOLUTION

□ STEP 5: Upload Profile Images via Admin

In students/admin.py:

: lab-6/rolasoft_school/students/admin.py

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'profile_image']
```

- Open admin panel (http://127.0.0.1:8000/admin/)
- Add a few students with profile images

LAB 6 ACTIVITY - SOLUTION

Open browser:
http://127.0.0.1:8000/admin/

If you already created a **superuser**, login.
If not, create one:
python manage.py createsuperuser

Example:
Username: admin
Email: admin@gmail.com
Password: ********

Then login at:
http://127.0.0.1:8000/admin/

Click Students → +Add

LAB 6 ACTIVITY - SOLUTION

□ STEP 6: Display Student Images in Template

Create students/templates/students/student_list.html:

: lab-6/rolasoft_solution/students/templates/students/student_list.html

□ STEP 7: Update Views

In students/views.py:

: lab-6/rolasoft_solution/students/views.py

□ STEP 8: Update URLs

In students/urls.py:

: lab-6/rolasoft_solution/students/urls.py

LAB 6 ACTIVITY - SOLUTION

□ STEP 9: Add Navigation Link

Update your navbar.html:

: lab-6/rolasoft_solution/students/templates/navbar.html

```
<li class="nav-item">
   <a class="nav-link" href="{% url 'student_list' %}">Students</a>
</li>
```

LAB 6 ACTIVITY - SOLUTION

□ STEP 10: Test Everything

1. **Run server:**

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-6\rolasoft_school>

python manage.py runserver

2. Visit homepage → see CSS applied and alert message
3. Visit /students/ → see student cards with images
4. Test uploading a new student with image in admin → refresh /students/ page

LAB 7 ACTIVITY

🏁 LAB 7 Class Work

1. Create Course model with title & description
2. Create Student model with name, email, enrollment_date
3. Add a profile_image field to Student
4. Create a Many-to-Many relationship: Student ↔ Course
5. Apply migrations and add 3 students + 2 courses via admin
6. Display students & courses in a template

We assume your Django project is **rolasoft_school** and your app is **students**.

LAB 7 ACTIVITY - SOLUTION

🔹 STEP 1: Create Models

Open students/models.py and define:

: lab-7/rolasoft_school/students/models.py

✅ What we did:

- Course → title & description
- Student → name, email, enrollment_date, profile_image
- courses = ManyToManyField(Course) → Student ↔ Course relationship

LAB 7 ACTIVITY - SOLUTION

□ STEP 2: Install Pillow (for ImageField)

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-7\rolasoft_school> pip install pillow

□ STEP 3: Apply Migrations

-:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-7\rolasoft_school>
python manage.py makemigrations students

Migration Question About enrollment_date

Message
It is impossible to add the field 'enrollment_date'
with 'auto_now_add=True' to student without providing a default
Why this happens

You added this field to your model:
enrollment_date = models.DateTimeField(auto_now_add=True)

But your database already contains students, so Django needs a value for the existing rows.

LAB 7 ACTIVITY - SOLUTION

✓ **Correct Option to Choose**

When Django asks:

Select an option:

1) Provide a one-off default now

2) Quit and manually define a default value

Choose: 1

Then press Enter.

Django will ask for a default value.

Just type: timezone.now

or press Enter to accept the current time.

-:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-7\rolasoft_school>

python manage.py migrate

This creates the tables in your database.

LAB 7 ACTIVITY - SOLUTION

✅ STEP 4: Register Models in Admin

Open students/admin.py:

: lab-7/rolasoft_school/students/admin.py

```
from django.contrib import admin
from .models import Student, Course

@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ['title', 'description']

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'email', 'enrollment_date', 'profile_image']
    filter_horizontal = ['courses']  # Makes ManyToMany selection easier
```

- **filter_horizontal** → allows easy multi-select for courses in admin

LAB 7 ACTIVITY - SOLUTION

? STEP 5: Add Sample Data via Admin

1. **Run server:**

python manage.py runserver

2. Go to: http://127.0.0.1:8000/admin/
3. Create 2 courses:

Title	Description
Python Programming	Learn Python from scratch
Django Web Development	Build web apps with Django

4. Create 3 students, assign courses, optionally upload profile images.

LAB 7 ACTIVITY - SOLUTION

Create Courses and Students Using Admin

Task

1. Go to /admin/
2. Create 2 Courses
3. Create 3 Students
4. Assign courses to students
5. Optionally upload profile images

Step 1 — Start the Django Server

Run:

python manage.py runserver

Open browser:

http://127.0.0.1:8000/admin/

LAB 7 ACTIVITY - SOLUTION

Step 2 — Login to Django Admin

If you already created a superuser, login.

If not, create one:

python manage.py createsuperuser

Example:

Username: admin

Email: admin@gmail.com

Password: ********

Then login at:

http://127.0.0.1:8000/admin/

LAB 7 ACTIVITY - SOLUTION

Step 3 — Create 2 Courses

Click Courses → Add

Create the following:

Course 1

Course Name: Python Programming

Description: Learn Python fundamentals

Click Save.

Course 2

Course Name: Web Development

Description: Learn HTML, CSS, JavaScript, Django

Click Save.

Now you should have 2 courses in the database.

LAB 7 ACTIVITY - SOLUTION

Step 4 — Create 3 Students

Go to:

Students → Add

Student 1

Name: John Doe

Email: john@example.com

Courses: Python Programming

Enrollment Date: auto

Profile Image: optional

Click Save.

Student 2

Name: Mary Johnson

Email: mary@example.com

Courses: Web Development

Profile Image: optional

Click Save.

LAB 7 ACTIVITY - SOLUTION

Student 3

Name: David Smith

Email: david@example.com

Courses:

✓ Python Programming

✓ Web Development

Profile Image: optional

Click Save.

LAB 7 ACTIVITY - SOLUTION

Step 5 — Verify the Relationship

You should now see:

Courses

Course	Students
Python Programming	John, David
Web Development	Mary, David

This means Many-to-Many relationship works correctly.

LAB 7 ACTIVITY - SOLUTION

Expected Admin Output

Admin should display:

Courses

- Python Programming
- Web Development

Students

- John Doe
- Mary Johnson
- David Smith

Each student should have assigned courses.

LAB 7 ACTIVITY - SOLUTION

□ STEP 6: Create Views to Display Data

Open students/views.py:

: lab-7/rolasoft_school/students/views.py

```
from django.shortcuts import render
from .models import Student, Course

def student_course_list(request):
    students = Student.objects.all().prefetch_related('courses')
    courses = Course.objects.all().prefetch_related('students')
    return render(request, 'students/student_course_list.html', {
        'students': students,
        'courses': courses
    })
```

- **prefetch_related** → optimizes ManyToMany queries

LAB 7 ACTIVITY - SOLUTION

□ STEP 7: Update URLs

Open students/urls.py:

: lab-7/rolasoft_school/students/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('students-courses/', views.student_course_list, name='student_course_list'),
]
```

LAB 7 ACTIVITY - SOLUTION

□ STEP 8: Create Template

Create students/templates/students/student_course_list.html:

: lab-7/rolasoft_school/students/templates/students/student_course_list.html

□ STEP 9: Add Navigation Link (Optional)

In navbar.html:

: lab-7/rolasoft_school/students/templates/navbar.html

```
<li class="nav-item">
   <a class="nav-link" href="{% url 'student_course_list' %}">Students & Courses</a>
</li>
```

LAB 7 ACTIVITY - SOLUTION

🔍 STEP 10: Test Everything

1. **Run server:**

python manage.py runserver

2. **Visit:**

http://127.0.0.1:8000/students-courses/

✅ You should see:

- Student cards with profile images
- Each student listing the courses they belong to
- Each course listing the students enrolled

LAB 8 ACTIVITY

🏆 LAB 8: Class Work

1. Create Profile → One-to-One with User
2. Create Course → One-to-Many with Student
3. Create Student → Many-to-Many with Course
4. Display in template:
   - User → Profile info
   - Student → Courses
   - Course → Enrolled Students

We assume your Django project is **rolasoft_school** and your app is **students**.

LAB 8 ACTIVITY - SOLUTION

□? STEP 1: Models Setup (students/models.py)

: lab-8/rolasoft_school/students/models.py

✅ What we did:

- Profile → One-to-One with User
- Course → simple model
- Student → One-to-One with User and Many-to-Many with Course

🚀 Pro Tip (Very Important for Django Students)

Always run this sequence after changing models:

python manage.py makemigrations

python manage.py migrate

LAB 8 ACTIVITY - SOLUTION

□ STEP 2: Models Setup (students/forms.py)

: lab-8/rolasoft_school/students/forms.py

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['user', 'courses']
```

LAB 8 ACTIVITY - SOLUTION

□ STEP 3: Install Pillow for ImageField

pip install pillow

□ STEP 4: Apply Migrations

python manage.py makemigrations students
python manage.py migrate

LAB 8 ACTIVITY - SOLUTION

□ STEP 5: Admin Setup (students/admin.py)

: lab-8/rolasoft_school/students/admin.py

```
from django.contrib import admin
from .models import Profile, Student, Course

@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    list_display = ['user', 'bio', 'profile_image']
@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ['title', 'description']

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['user', 'enrollment_date']
    filter_horizontal = ['courses']  # Easy multi-select for Many-to-Many
```

LAB 8 ACTIVITY - SOLUTION

□ STEP 6: Views (students/views.py)

: lab-8/rolasoft_school/students/views.py

Add:

```
from django.shortcuts import render
from django.contrib.auth.models import User
from .models import Profile, Student, Course

def dashboard(request):
    users = User.objects.all().prefetch_related('profile', 'student__courses')
    students = Student.objects.all().prefetch_related('courses')
    courses = Course.objects.all().prefetch_related('students')

    return render(request, 'students/dashboard.html', {
        'users': users,
        'students': students,
        'courses': courses
    })
```

LAB 8 ACTIVITY - SOLUTION

□ STEP 7: URLs (students/urls.py)

: lab-8/rolasoft_school/students/urls.py

Add:

```
from django.urls import path
from . import views

urlpatterns = [
    path('dashboard/', views.dashboard, name='dashboard'),
]
```

□ STEP 8: Template (students/templates/students/dashboard.html)

: lab-8/rolasoft_school/students/templates/students/dashboard.html

LAB 8 ACTIVITY - SOLUTION

□ STEP 9: Base Template (templates/base.html)

: lab-8/rolasoft_school/students/templates/base.html

□ STEP 10: Navbar (templates/navbar.html)

: lab-8/rolasoft_school/students/templates/navbar.html

Add:

```
<li class="nav-item"><a class="nav-link" href="/dashboard/">Dashboard</a></li>
```

LAB 8 ACTIVITY - SOLUTION

□ STEP 11: Testing

✓ (Clean Reset – Best for Lab Projects)

Step 1 — Stop Server

Press:

CTRL + C

Step 2 — Delete Database

Delete the file:

db.sqlite3

Step 3 — Delete Migrations

Go to:

students/migrations/

Delete everything except:

__init__.py

LAB 8 ACTIVITY - SOLUTION

Step 4 — Recreate Migrations

Run:

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-8\rolasoft_school>

python manage.py makemigrations

Step 5 — Rebuild Database

Run:

python manage.py migrate

You will see many migrations applied.

Step 6 — Create Admin User Again

python manage.py createsuperuser

Example:

Username: admin

Email: admin@gmail.com

Password: ********

LAB 8 ACTIVITY - SOLUTION

Step 7 — Run Server

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-8\rolasoft_school>

python manage.py runserver

Open:

http://127.0.0.1:8000/admin

1. Go to /admin/:
   - Add Users
   - Create Profiles for users
   - Add Courses
   - Add Students linked to Users and assign Courses
2. Visit /dashboard/ → see:
   - Users → Profile info
   - Students → Enrolled Courses
   - Courses → Enrolled Students

✓ All relationships are displayed correctly in templates.

LAB 9 ACTIVITY

🏆 LAB 9: Class Work

Django ORM queries in action, with:

- Models (User, Profile, Student, Course)
- Admin setup
- Views using ORM queries (all, filter, exclude, annotate, aggregate, select_related, prefetch_related)
- Templates showing query results
- Pre-filled sample data (via fixture)
- Navbar + Bootstrap
- Static & media files

LAB 9 ACTIVITY - SOLUTION

1) Models (students/models.py)

: lab-9/rolasoft_school/students/models.py

2)Admin Setup (students/admin.py)

: lab-9/rolasoft_school/students/admin.py

3) Views (students/views.py)

: lab-9/rolasoft_school/students/views.py

LAB 9 ACTIVITY - SOLUTION

4) URLs

students/urls.py

: lab-9/rolasoft_school/students/urls.py

rolasoft_school/urls.py

: lab-9/rolasoft_school/rolasoft_school/urls.py

LAB 9 ACTIVITY - SOLUTION

□ 5) Templates

templates/base.html

: lab-9/rolasoft_school/students/templates/base.html

templates/navbar.html

: lab-9/rolasoft_school/students/templates/navbar.html

templates/students/home2.html

: lab-9/rolasoft_school/students/templates/students/home2.html

```
{% extends "base.html" %}
{% block title %}Home{% endblock %}
{% block content %}
<div class="text-center">
    <h1 class="display-4">Welcome to Django ORM Demo</h1>
    <p class="lead">Explore ORM queries in action</p>
    <a href="{% url 'orm_demo' %}" class="btn btn-primary btn-lg">Go to ORM Demo</a>
</div>   {% endblock %}
```

LAB 9 ACTIVITY - SOLUTION

templates/students/orm_demo.html

: lab-9/rolasoft_school/students/templates/students/orm_demo.html

□ 6) Static Files

static/css/style.css

```
body { font-family: Arial, sans-serif; }
h1.display-4 { color: #007bff; }
```

static/js/script.js

```
window.onload = function() {
    console.log("ORM Demo Loaded");
}
```

□ 7) Sample Data Fixture (students/fixtures/initial_data.json)

: lab-9/rolasoft_school/students/fixtures/initial_data.json

LAB 9 ACTIVITY - SOLUTION

8) Run the Project

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-9\rolasoft_school>

pip install django pillow
python manage.py makemigrations
python manage.py migrate
python manage.py loaddata students/fixtures/initial_data.json
python manage.py createsuperuser
python manage.py runserver

- Visit / → Homepage
- Visit /orm-demo/ → See ORM queries in action: all students, filtered lists, counts, aggregates, complex queries

LAB 10 ACTIVITY

□ LAB 10: Class Work

1. Create a Contact Form with forms.Form
2. Create a Student Form using forms.ModelForm
3. Add validation to ensure the name has no numbers
4. Add a profile image upload field
5. Render all forms in Bootstrap-styled templates

LAB 10 ACTIVITY - SOLUTION

Step 1: Create Forms (students/forms.py)

: lab-10/rolasoft_school/students/forms.py

Step 2: Create Views (students/views.py)

: lab-10/rolasoft_school/students/views.py

LAB 10 ACTIVITY - SOLUTION

🏫 Step 3: URLs (students/urls.py)

: lab-10/rolasoft_school/students/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('contact/', views.contact_view, name='contact'),
    path('add-student/', views.add_student_view, name='add_student'),
]
```

Include in rolasoft_school/urls.py:

: lab-10/rolasoft_school/rolasoft_school/urls.py

```
path('', include('students.urls')),
```

LAB 10 ACTIVITY - SOLUTION

🧰 Step 4: Templates with Bootstrap

templates/navbar.html

: lab-10/rolasoft_school/students/templates/navbar.html

```
<li class="nav-item"><a class="nav-link" href="{% url 'contact' %}">Contact</a></li>
<li class="nav-item"><a class="nav-link" href="{% url 'add_student' %}">Add Student</a></li>
```

templates/students/contact.html

: lab-10/rolasoft_school/students/templates/students/contact.html

templates/students/add_student.html

: lab-10/rolasoft_school/students/templates/students/add_student.html

LAB 10 ACTIVITY - SOLUTION

Step 5: Settings for Media Upload (rolasoft_school/settings.py)

: lab-10/rolasoft_school/rolasoft_school/settings/base.py

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Add to urls.py for development:

: lab-10/rolasoft_school/rolasoft_school/urls.py

```
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

LAB 10 ACTIVITY - SOLUTION

🧰 Step 6: Test the Project

1. **Run Migrations:**

:cd \Python_Diploma_V2\DJANGO_PROJECT\LAB-10\rolasoft_school>

pip install pillow

python manage.py makemigrations

python manage.py migrate

2. **Create superuser (to select users for StudentForm):**

python manage.py createsuperuser

3. **Run Server:**

python manage.py runserver

4. **Visit:**

- /contact/ → Test Contact Form with validation
- /add-student/ → Test Student Form + profile image upload

LAB 11 ACTIVITY - SOLUTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 7 ACTIVITY - SOLUTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 7 ACTIVITY - SOLUTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 3 ACTIVITY - Modify your project

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 1 ACTIVITY -Django Activity

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 1 ACTIVITY -Django Activity

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 1 ACTIVITY -Django Activity

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ADVANCED DJANGO DEVELOPMENT

ADVANCED DJANGO DEVELOPMENT

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ADVANCED DJANGO DEVELOPMENT

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

VERSION CONTROL & COLLABORATION

VERSION CONTROL & COLLABORATION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

VERSION CONTROL & COLLABORATION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

SECURITY & BEST PRACTICES

SECURITY & BEST PRACTICES

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

SECURITY & BEST PRACTICES

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DEPLOYMENT & PRODUCTION

DEPLOYMENT & PRODUCTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DEPLOYMENT & PRODUCTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FINAL CAPSTONE PROJECT

FINAL CAPSTONE PROJECT

Project Options

- School ERP System
- Hospital Management System
- E-Commerce Platform
- HR Management System
- Multi-Tenant SaaS Application

Requirements

- Authentication & authorization
- Role-based access
- CRUD operations
- REST APIs
- Responsive UI
- Live deployment
- GitHub repository

FINAL CAPSTONE PROJECT

Outcome

✓ Portfolio-ready real-world application

✓ Job & internship readiness

📜 OPTIONAL CERTIFICATIONS

- Python Certification
- Django Developer Certification
- PostgreSQL Certification
- Git & GitHub Certification

FINAL CAPSTONE PROJECT

SKILLS AFTER COMPLETION

✓ Python Programming
✓ Django Web Development
✓ Frontend (HTML, CSS, JS)
✓ Database Design & ORM
✓ REST API Development
✓ Authentication & Security
✓ Deployment & DevOps Basics

CAREER PATHS

- Django Backend Developer
- Full-Stack Developer
- Python Software Engineer
- Web Application Developer
- Junior DevOps Engineer

Capstone Project: End-to-End Web Application using Django + Database

End Of Django Web Development!